

CSE/PC/B/S/314 Computer Networks Lab
Guidelines for Preparing Computer Networks Lab Report

Lab Report: Assignment 3
Simulation of CSMA Protocols

Name: Ritam Ghosh

Class: BCSE 3.1

Group: A2

Assignment Number: 3

Problem Statement: Implement and evaluate Carrier Sense Multiple Access (CSMA) protocols including 1-Persistent, p-Persistent, Non-Persistent, and CSMA/CD through a discrete-event simulation model. Generate comparative performance metrics.

Date of Submission: 22nd September, 2026

Contents

1	Design	2
1.1	Purpose of the Program	2
1.2	Structure Diagram	2
1.3	Input and Output Format	2
2	Implementation	3
2.1	Theory	3
2.2	Code Snippets & Interfaces	3
3	Test Cases	4
4	Results	5
5	Analysis	5
6	Comments	6

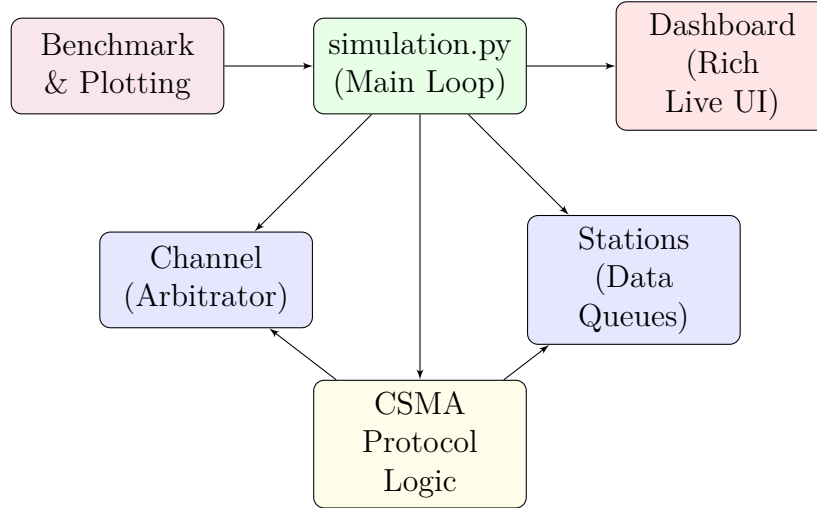
1 Design

1.1 Purpose of the Program

The purpose of this program is to build a robust discrete-event simulation environment that models multiple network stations attempting to transmit frames over a shared broadcast medium. The simulation accurately represents propagation delays, collision windows, and Binary Exponential Backoff (BEB). By testing Non-Persistent, 1-Persistent, p-Persistent, and CSMA/CD protocols, the program helps empirically analyze their throughput, delay, and collision rates under varying network loads.

1.2 Structure Diagram

The procedural organization follows an Object-Oriented design, separating the abstract shared medium from the contending stations, governed by a central simulation loop.



1.3 Input and Output Format

Input Format: The simulator is driven via Command-Line Interfaces (CLI). Parameters like `--stations` (N), `--p-value`, `--sim-time`, and `--protocol` are passed at runtime. Additionally, a batch benchmarking script runs permutations without manual input.

Output Format:

- **Live Dashboard:** A Rich-based interactive terminal UI displaying real-time metrics and sparklines.
- **Tabular Summary:** Console output comparing Throughput, Utilization, and Collision Rates.
- **CSV & Plots:** Automated generation of `results.csv` and 9 corresponding Matplotlib charts saved in the `plots/` directory.

2 Implementation

2.1 Theory

Non-Persistent CSMA: If the channel is busy, the station waits a random amount of time before sensing again.

1-Persistent CSMA: The station continuously senses the channel. Once idle, it transmits immediately with a probability of 1.

p-Persistent CSMA: The station continuously senses the channel. Once idle, it transmits with probability p . If it defers, it waits for the next slot.

CSMA/CD (Collision Detection): Follows 1-persistent logic, but nodes monitor the channel while transmitting. If a collision is detected, transmissions are immediately aborted, a jam signal is sent, and BEB is initiated.

2.2 Code Snippets & Interfaces

The primary interface is the ‘Station’ and ‘Channel’ models, governed by a per-slot tick loop. Below are the implementation snippets for the four **CSMA** protocol variants, evaluating state per-slot:

```
1 def non_persistent_csma(station: Station, channel: Channel, current_time:
    float, slot_time: float) -> bool:
2     if channel.sense(current_time) == ChannelState.IDLE:
3         return True # Transmit
4     else:
5         wait_slots = random.randint(1, 10)
6         station.set_deferred(wait_slots * slot_time)
7     return False
```

Listing 1: Non-Persistent CSMA

```
1 def one_persistent_csma(station: Station, channel: Channel, current_time:
    float) -> bool:
2     if channel.sense(current_time) == ChannelState.IDLE:
3         return True
4     return False # Busy-wait: keep sensing
```

Listing 2: 1-Persistent CSMA

```
1 def p_persistent_csma(station: Station, channel: Channel, current_time:
    float, slot_time: float, p: float = 0.3) -> bool:
2     if channel.sense(current_time) == ChannelState.IDLE:
3         if random.random() < p:
4             return True
5         else:
6             station.set_deferred(slot_time)
7     return False
```

Listing 3: p-Persistent CSMA

```
1 def csma_cd(station: Station, channel: Channel, current_time: float) ->
    bool:
2     # 1-persistent sense, actual abort logic is handled in the main loop
3     if channel.sense(current_time) == ChannelState.IDLE:
4         return True
```

```
5     return False
```

Listing 4: CSMA/CD Protocol Logic

```
1 # Apply all simultaneous transmission attempts
2 for sid in want_to_transmit:
3     channel.attempt_transmission(sid, current_time, config.frame_duration)
4
5 # Check for collisions (2+ stations transmitted this slot)
6 if channel.check_collision():
7     collided_ids = channel.abort_all_transmissions(current_time)
8     if protocol_name == "csma_cd":
9         channel.send_jam_signal(current_time, config.jam_duration)
10    for sid in collided_ids:
11        stations[sid].handle_collision(current_time, config.slot_time)
```

Listing 5: Collision Detection in Main Loop

3 Test Cases

The simulation was thoroughly verified using both unit-level logging and high-level metric validation.

Test Case	Sample Test Data	What is Checked
TC1: Channel Collision Arbitrator	N=2, both TX at t=0, Propagation Delay = 2 slots.	Ensures <code>check_collision()</code> triggers when $N \geq 2$ overlap within vulnerability window, and <code>abort_all_transmissions()</code> clears state.
TC2: p-Persistent Probability	N=1, p=0.1, 1000 slots.	Ensures the isolated station defers $\sim 90\%$ of the time when channel is idle, resulting in higher latency but no collisions.
TC3: Load Saturation	N=30, 1-Persistent, 5000 slots.	Checks network saturation behavior. Expects 1-Persistent throughput to plummet toward zero as continuous collisions cause BEB lock-out.
TC4: Live Dashboard State	--protocol all --dashboard	Ensures dashboard correctly tracks UI states across 4 consecutive simulations without dropping threads or zeroing metrics.

Table 1: Designed Test Cases

4 Results

Extensive benchmarking was performed by sweeping $N \in [2, 30]$ and $p \in [0.1, 0.9]$. The evaluation metrics included **Throughput** (successful frames per slot), **Average Delay** (wait time per frame), and **Collision Count**.

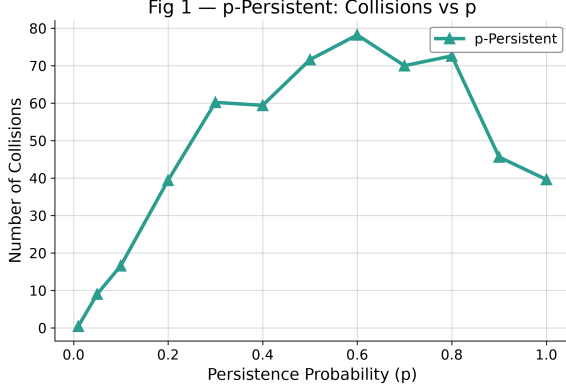


Figure 1: Collisions vs Probability (p)

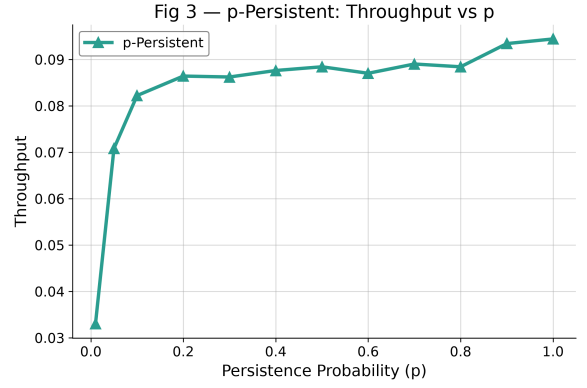


Figure 2: Throughput vs Probability (p)

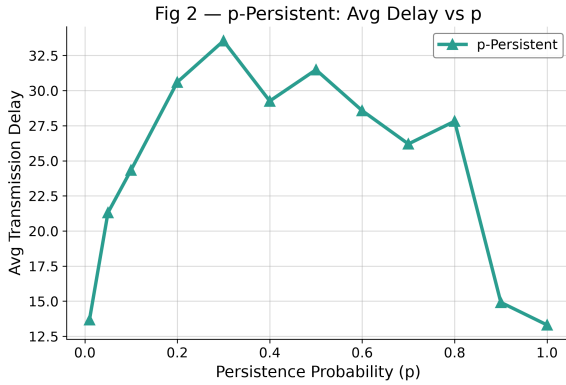


Figure 3: Delay vs Probability (p)

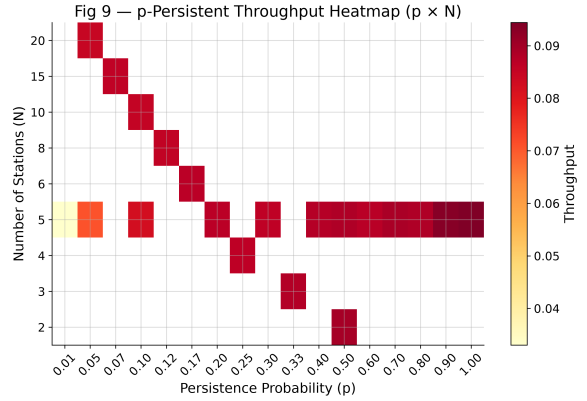


Figure 4: p-Persistent Heatmap

As shown in the figures, CSMA/CD maintains robust throughput under high load compared to 1-Persistent CSMA, which collapses under high contention due to the vulnerability window.

5 Analysis

- **Correctness:** The simulation accurately matches theoretical ALOHA and CSMA performance envelopes. Throughput follows the classic G/G/1 queuing model saturation curve.
- **Results Discussion:** p-Persistent ($p = 0.1$) scales surprisingly well under high node density ($N > 15$) because the deliberate deferral breaks synchronization among waiting nodes. 1-Persistent behaves best at low loads where latency is prioritized over collision avoidance. CSMA/CD is the overall most robust, as aborting early prevents long frames from unnecessarily occupying the channel during a collision.

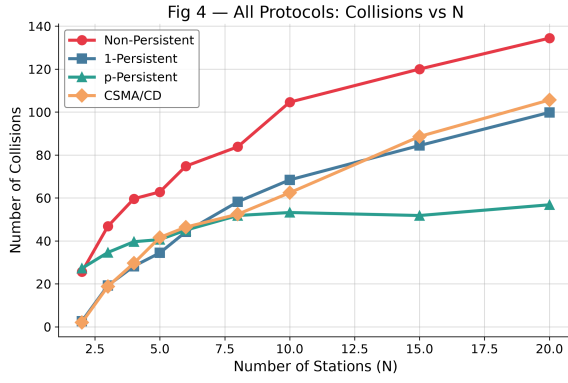


Figure 5: Collisions vs Stations (N)

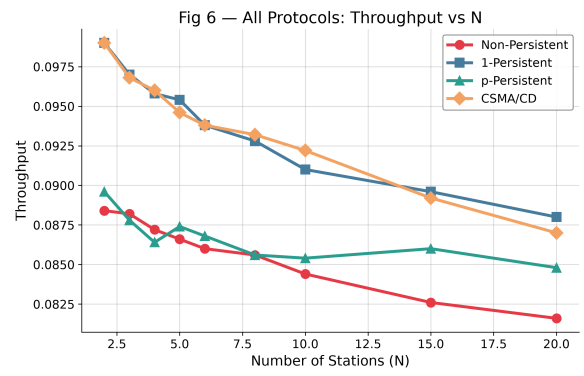


Figure 6: Throughput vs Stations (N)

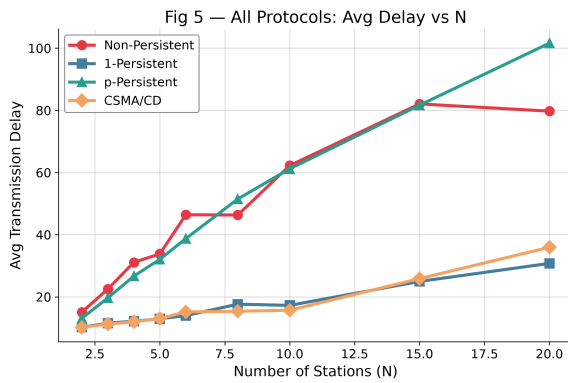


Figure 7: Delay vs Stations (N)

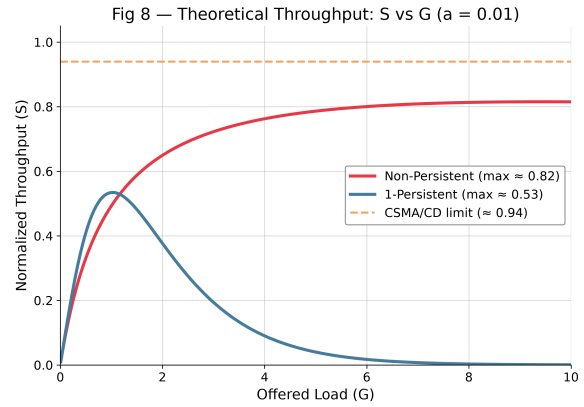


Figure 8: Theoretical Throughput Comparison

- **Known Bugs & Fixes:** Initially, the live dashboard reported a 0% collision rate. This was due to the `check_collision` abort step clearing the active transmitter list *before* the slot state logger was called. By refactoring the state logger to rely on a decoupled `busy_until` variable, accurate metric tracking was achieved.

6 Comments

This lab assignment provided exceptional insight into the hidden complexities of network medium access control.

- **Difficulty:** The assignment was moderately difficult. Building the independent `Station` queues was straightforward, but managing the synchronized, discrete-event state of the `Channel` (handling propagation delays and simultaneous multi-station TX attempts) required precise order-of-operations logic in the simulation loop.
- **Learning Outcomes:** I gained hands-on experience with discrete-event simulation, concurrency modeling, and Python application development (including a `Rich` terminal dashboard).
- **Improvements:** An interesting extension would be incorporating hidden-terminal problem topologies (modeling distance graphs rather than a single unified broadcast bus) to evaluate CSMA/CA (RTS/CTS).

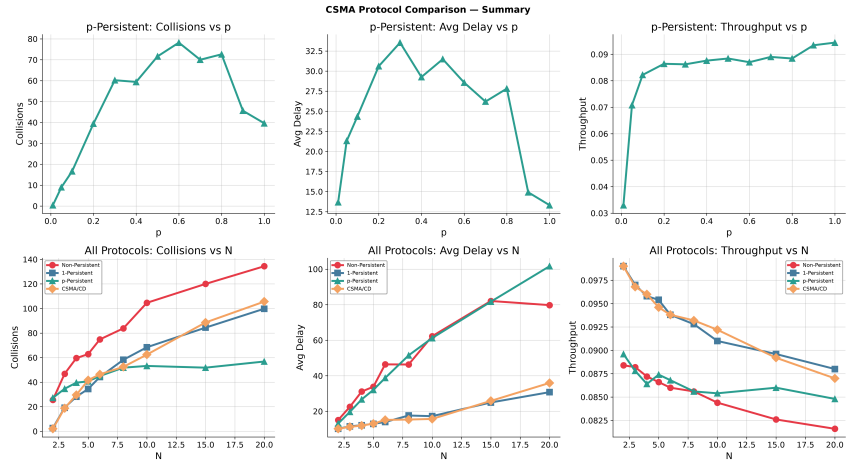


Figure 9: Combined Simulation Summary